一、 日志分析与文本处理 (高频必考)

这是后端开发日常最频繁的操作，也是面试重灾区。

1. 实时查看日志 (`tail`)

- **场景**：生产环境报错，需要实时监控 `app.log` 日志文件的最新输出，看是否有新的 Exception 产生。
- **命令**：

  Bash

  ```
  tail -f app.log
  ```

- **讲解**：`tail` 默认显示文件末尾 10 行。加上 `-f` (follow) 会挂起进程，持续输出文件新增加的内容。
- **参数拓展**：
  - `-n 100`：显示末尾 100 行（`tail -n 100 -f app.log`）。
  - `tail -F` (大写)：与 `-f` 类似，但如果文件被重命名或删除后重建（如日志轮转），`-F` 会尝试重新打开文件，而 `-f` 会停止。

2. 关键词搜索 (`grep`)

- **场景**：在几 G 的日志文件中，快速查找包含 “NullPointerException” 的报错行，并查看报错的前后上下文。
- **命令**：

  Bash

  ```
  grep -C 5 "NullPointerException" app.log
  ```

- **讲解**：`grep` 是全局正则表达式打印。它是文本搜索的神器。
- **参数拓展**：
  - `-i`：忽略大小写。
  - `-v`：反向查找（显示**不**包含匹配文本的行）。
  - `-n`：显示行号。

- `-A 5` (After)：显示匹配行及之后的 5 行。
- `-B 5` (Before)：显示匹配行及之前的 5 行。
- `-C 5` (Context)：显示匹配行前后各 5 行（面试加分项）。
- `-c`：统计匹配的行数（不显示内容，只给数字）。

3. 复杂数据提取与统计 (`awk`)

- **场景**：分析 Nginx 访问日志，统计访问量最高的 Top 10 IP 地址。
- **命令**：

  Bash

  ```
  awk '{print $1}' access.log | sort | uniq -c | sort -nr | head -n 10
  ```

- **讲解**：`awk` 是一门文本处理语言。默认按空格/Tab 分割。`$1` 代表第一列（通常是 IP）。
  - 组合拳逻辑：提取IP -> 排序 -> 去重并统计次数 -> 按次数倒序排列 -> 取前10。
- **参数拓展**：
  - `-F ':'`：指定分隔符（例如处理 `/etc/passwd` 时用冒号分割）。
  - `NR`：行号（Current Record Number）。
  - `NF`：列数（Number of Fields）。
  - `awk 'NR>100 {print $0}'`：打印第 100 行之后的内容。

4. 大文件分页查看 (`less`)

- **场景**：日志文件高达 5GB，用 `cat` 会刷屏卡死，需要翻页查看。
- **命令**：

  Bash

  ```
  less large_log.log
  ```

- **讲解**：`less` 是 `more` 的增强版，支持前后翻页，且加载速度快（不是一次性加载整个文件）。
- **操作拓展**：
  - `G` (Shift+g)：直接跳到文件末尾。

- `g`：直接跳到文件开头。
- `/keyword`：向下搜索关键词（按 `n` 查找下一个）。
- `?keyword`：向上搜索关键词。

二、 系统监控与进程管理 (性能排查)

当服务器变慢、CPU 飙高时，这类命令是救命稻草。

5. 综合系统监控 (`top`)

- **场景**：服务器响应极慢，需要查看是哪个进程占用了过多的 CPU 或内存，以及系统的负载情况。
- **命令**：

  Bash

  ```
  top
  ```

- **讲解**：实时显示系统中各个进程的资源占用状况。
  - **重点关注**：`Load Average` (1分钟, 5分钟, 15分钟负载)、`%CPU`、`%MEM`。
- **交互拓展** (进入 top 界面后)：
  - `P` (大写)：按 CPU 使用率排序。
  - `M` (大写)：按内存使用率排序。
  - `1` (数字)：显示所有 CPU 核心的详细情况。
  - `c`：显示完整的命令行路径（不仅仅是进程名）。

6. 查看特定进程信息 (`ps`)

- **场景**：确认 Java 服务是否启动，或者查找僵尸进程。
- **命令**：

  Bash

  ```
  ps -ef | grep java
  ```

  或者

  Bash

  ```
  ps aux | grep java
  ```

- **讲解**：`ps` (process status) 用于显示当前进程的状态（静态快照）。
- **参数拓展**：
  - `-e`：显示所有进程。
  - `-f`：全格式显示（包含 UID, PID, PPID, C, STIME, TTY, TIME, CMD）。
  - `aux`：BSD 风格，`a` 显示所有用户的进程，`u` 显示用户/内存/CPU信息，`x` 显示无控制终端的进程。

7. 端口占用与网络连接 (`netstat` / `ss`)

- **场景**：启动服务报错 “Address already in use”，需要查出哪个进程占用了 8080 端口。
- **命令**：

  Bash

  ```
  netstat -tulpn | grep 8080
  ```

- **讲解**：查看网络连接、路由表、接口统计等。
- **参数拓展 (背诵口诀：tulpn)**：
  - `-t`：TCP 协议。
  - `-u`：UDP 协议。
  - `-l`：Listening (监听状态)。
  - `-p`：显示 PID 和进程名称 (需要 sudo 权限)。
  - `-n`：Numeric (直接显示 IP 和端口号，不解析域名，速度快)。
  - *注*：`ss` 命令是 `netstat` 的现代替代品，参数几乎一样 (`ss -tulpn`)，在大并发下性能更好。

8. 查看文件打开情况 (`lsof`)

- **场景**：想删除一个文件，但提示“文件忙”；或者想看某个进程到底打开了哪些文件/网络连接。
- **命令**：

  Bash

  ```
  lsof -i :8080
  ```

- **讲解**：List Open Files。在 Linux 中一切皆文件，它不仅能看文件，还能看网络端口。
- **参数拓展**：
  - `-i :端口`：查看谁在用这个端口。
  - `-p PID`：查看某个 PID 进程打开了哪些文件。
  - `+D /path`：查看谁在使用该目录下的文件。

三、 磁盘与内存管理

9. 查看磁盘空间 (`df`)

- **场景**：应用报错 “No space left on device”，检查磁盘挂载点使用情况。
- **命令**：

  Bash

  ```
  df -h
  ```

- **讲解**：Disk Free。显示磁盘分区上的可用的磁盘空间。
- **参数拓展**：
  - `-h`：Human readable (以 KB, MB, GB 为单位显示，便于阅读)。
  - `-i`：查看 Inode 使用率（有时磁盘没满但 Inode 满了也无法写入）。

10. 查看目录大小 (`du`)

- **场景**：磁盘满了，`df` 发现 `/var` 占用 100%，需要找出是哪个具体文件夹（如日志目录）占用的。
- **命令**：

  Bash

  ```
  du -sh *
  ```

- **讲解**：Disk Usage。显示每个文件和目录的磁盘使用空间。
- **参数拓展**：
  - `-s`：Summary (只显示总和，不列出子文件)。
  - `-h`：Human readable。

- `du -h --max-depth=1`：只查看当前目录下第一层子目录的大小。

11. 内存使用情况 (`free`)

- **场景**：检查服务器剩余内存，判断是否需要升级配置或排查内存泄漏。
- **命令**：

  Bash

  ```
  free -h
  ```

- **讲解**：显示内存的使用情况，包括物理内存、交换内存(swap)和内核缓冲区内存。
- **重点关注**：
  - `buff/cache`：Linux 会把空闲内存用于缓存，这部分在应用程序需要时是可回收的。
  - `available`：真正应用程序可用的内存估算值。

四、 网络工具与文件传输

12. 网络连通性测试 (`ping`)

- **场景**：测试服务器是否能连通外网，或两台内网机器是否互通。
- **命令**：

  Bash

  ```
  ping www.google.com
  ```

- **讲解**：基于 ICMP 协议，检测网络连通性和丢包率。
- **参数拓展**：
  - `-c 4`：只 ping 4 次后停止（默认会一直 ping）。

13. 接口测试与下载 (`curl`)

- **场景**：在服务器上排查接口问题，模拟发送一个 POST 请求，或者下载一个脚本。
- **命令**：

  Bash

```
curl -X POST -H "Content-Type: application/json" -d '{"id":1}'
http://api.example.com
```

- **讲解**：强大的文件传输工具，支持 HTTP, HTTPS, FTP 等。
- **参数拓展**：
  - `-I` (大写 i)：只抓取 HTTP Header，不下载 Body (用于查看 200/404 状态码)。
  - `-o filename`：将输出保存为文件。
  - `-L`：跟随重定向 (Follow Redirects)。
  - `-v`：Verbose (显示详细握手过程，Debug 神器)。

五、 权限与查找

14. 修改权限 (`chmod`)

- **场景**：部署脚本 `run.sh` 提示 “Permission denied”。
- **命令**：

  Bash

  ```
  chmod +x run.sh
  ```

- **讲解**：Change Mode。
- **参数拓展**：
  - `777`：所有权限（读+写+执行，慎用）。
  - `755`：所有者可读写执行，其他人可读执行。
  - `-R`：递归修改目录下所有文件。

15. 文件查找 (`find`)

- **场景**：忘记配置文件放在哪了，只记得名字里包含 “nginx”；或者需要删除 7 天前的日志文件。
- **命令**：

  Bash

  ```
  find / -name "*nginx*"
  ```

- **讲解**：在目录树中搜索文件，功能极其强大。

- **参数拓展**：
  - `-name`：按文件名查找。
  - `-type f`：只找文件；`-type d`：只找目录。
  - `-mtime +7`：查找修改时间在 7 天以前的文件。
  - `-size +100M`：查找大于 100MB 的文件。
  - **组合技**：`find /var/log -name "*.log" -mtime +7 -delete` (自动删除7天前的日志)。

以下是 5 个高难度的“组合拳”场景，专注于 **Log 分析、批量操作和性能统计**。

一、 高频日志统计：找出攻击者的 IP

- **场景**：Nginx 访问日志 (`access.log`) 巨大，发现大量 404 错误，怀疑有人在扫描路径。请找出产生 404 错误最多的前 5 个 IP 地址，并显示其出现次数。
- **命令**：

  Bash

  ```
  grep "404" access.log | awk '{print $1}' | sort | uniq -c | sort -nr | head -n 5
  ```

- **深度讲解**：这是一条经典的“日志分析流水线”，每个环节缺一不可：
  1. `grep "404" access.log`：先过滤，只保留包含 404 的行（减少后续数据处理量）。
  2. `awk '{print $1}'`：提取第一列（通常 Nginx 日志第一列是客户端 IP）。
  3. `sort`：**关键步骤**。`uniq` 去重统计的前提是数据必须相邻，所以必须先排序，把相同的 IP 排在一起。
  4. `uniq -c`：去重并计数 (Count)。输出格式变为：“次数 IP”。
  5. `sort -nr`：再次排序。`-n` 按数值排序（而不是字符），`-r` 倒序（从大到小）。
  6. `head -n 5`：取前 5 名。
- **参数/变种拓展**：
  - 如果日志很大（GB级），`cat` 可能会卡死，建议用 `tail -n 10000` 先取一部分样本分析。

- **面试加分点**：如果是查看 502 报错，且日志带时间戳，可以 `cut` 截取时间字段，统计“每分钟的报错数量”，画出流量波形。

二、 批量“杀”进程：清理僵死服务

- **场景**：服务器上启动了多个 `php-fpm` 或者 `worker` 进程，现在需要全部重启，但不知道具体的 PID，只知道进程名包含 “worker”。如何**一次性**杀掉它们?
- **命令**：

  Bash

  ```
  ps -ef | grep "worker" | grep -v "grep" | awk '{print $2}' | xargs kill -9
  ```

- **深度讲解**：
  1. `ps -ef | grep "worker"`：找出所有含 “worker” 字符的进程。
  2. `grep -v "grep"`：**经典坑点**。第一步的 `grep` 命令本身也会被 `ps` 抓到，需要把它反向过滤掉，否则自己杀自己会报错。
  3. `awk '{print $2}'`：提取第二列，即进程的 **PID**。
  4. `xargs kill -9`：**核心**。`awk` 输出的是一堆数字（标准输出），而 `kill` 命令需要参数。`xargs` 的作用就是**把标准输出转换成命令行参数**，相当于执行了 `kill -9 PID1 PID2 PID3...`。
- **参数/变种拓展**：
  - `pgrep -f "worker" | xargs kill -9`：这是简写版，`pgrep` 专门用于查找进程 PID，省去了 `ps` 和 `grep` 的麻烦。面试时写上面长版本展示逻辑，写这个版本展示效率。

三、 TCP 连接状态统计：排查连接泄露

- **场景**：服务器负载很高，怀疑有连接泄露（Connection Leak）或者被 DDoS 攻击。请统计当前 TCP 连接中，各个状态（ESTABLISHED, TIME_WAIT, SYN_RECV 等）的数量各是多少。
- **命令**：

  Bash

  ```
  netstat -n | awk '/^tcp/ {++S[$NF]} END {for(a in S) print a, S[a]}'
  ```

- **深度讲解**：

- 这是一个**纯 awk 脚本**的高级用法，不需要 `sort` 和 `uniq`，效率极高。
- `netstat -n`：列出所有网络连接。
- `/^tcp/`：awk 的正则匹配，只处理以 “tcp” 开头的行（过滤掉表头）。
- `{++S[$NF]}`：构建一个关联数组 `S`。`$NF` 表示当前行的**最后一列**（即状态列，如 ESTABLISHED）。这行代码意思是：遇到一个状态，就在数组里对应的计数器加 1。
- `END {for(a in S) print a, S[a]}`：处理完所有行后，遍历数组 `S`，打印状态名 (`a`) 和对应的计数值 (`S[a]`)。

- **参数/变种拓展**：
  - 如果发现 `TIME_WAIT` 也就是等待关闭的连接数达到几万，说明服务器主动关闭了大量连接，可能需要优化内核参数 `tcp_tw_reuse`。
  - 如果 `SYN_RECV` 很高，可能正在遭受 SYN Flood 攻击。

四、 文本“外科手术”：批量替换文件内容

- **场景**：项目迁移，需要把 `/var/www/html` 目录下所有 `.conf` 配置文件中的数据库地址 `192.168.1.100` 替换为 `10.0.0.5`。
- **命令**：

  Bash

  ```
  grep -rl "192.168.1.100" /var/www/html | xargs sed -i
  's/192.168.1.100/10.0.0.5/g'
  ```

- **深度讲解**：
  1. `grep -r` (Recursive)：递归查找。
  2. `-l` (List file only)：**关键参数**。只输出包含该字符串的**文件名**，而不是输出文件内容。
  3. `xargs`：接收前面的文件名列表，传给后面的命令。
  4. `sed -i`：Stream Editor。`-i` 表示**原地修改**（In-place edit），直接修改文件，不输出到屏幕。
  5. `'s/old/new/g'`：sed 的标准替换语法，`g` 代表全局替换（一行有多个也全部替换）。

- **参数/变种拓展**：

- **安全提示**：在执行 `sed -i` 之前，建议先去掉 `-i` 执行一次，看看输出结果是否符合预期，防止改错文件。

五、 计算接口平均响应时间 (Awk 算术)

- **场景**：Access Log 格式如下，最后一列是**接口响应耗时**（单位秒）：

  ```
  [Time] [IP] [URL] [Status] [ResponseTime]

  2023-10-01 10:00:00 1.2.3.4 /api/user 200 0.05
  ```

  请计算 `/api/user` 接口的平均响应时间。

- **命令**：

  Bash

  ```
  grep "/api/user" access.log | awk '{sum+=$NF; count++} END {print "Avg=", sum/count}'
  ```

- **深度讲解**：
  - awk 不仅仅是文本处理，它还是一门编程语言，支持变量和算术运算。
  - `sum+=$NF`：创建一个变量 `sum`，累加每一行的最后一列（耗时）。
  - `count++`：计数器加 1。
  - `END`：最后输出总和除以总数。

- **参数/变种拓展**：
  - 如果要过滤掉异常慢的请求（比如大于 1秒），可以在 awk 里加 `if` 判断：

    ```
    awk '$NF < 1 {sum+=$NF...}'
    ```

面试官的“心理博弈”

当你写出这些命令时，面试官通常会追问一个问题：

> “这些命令在日志文件非常大（比如 100GB）的时候会有什么问题？”

标准答案：

1. **IO 瓶颈**：管道流虽然是流式的，但大量的读写依然消耗 IO。
2. **内存问题**：`sort` 命令在处理海量数据时，如果内存不够，会使用磁盘临时文件，导致速度极慢。

3. **解决方案**：
    - 使用 `awk` 替代 `sort | uniq`（利用内存哈希表，速度更快，参考案例三）。
    - 或者明确回答：“生产环境大日志不建议在本地 Shell 统计，应该接入 ELK (Elasticsearch, Logstash, Kibana) 或 ClickHouse 进行分析。”